

PARTE A

```
#include <msp430g2553.h>
```

```
// DEFINE
```

```
#define SM_Hardware_Setup (0)
```

```
#define SM_Led_Blink (1)
```

```
#define LED_VD (BIT0)
```

```
// DECLARACAO DAS VARIAVEIS
```

```
unsigned char Status_Machine = SM_Hardware_Setup;
```

```
unsigned int Led_Blink_Tmr = 0;
```

```
// DECLARACAO DOS PROTOTIPOS
```

```
void Hardware_Setup (void);
```

```
void Oscilator_Setup (void);
```

```
void Timer_Setup (void);
```

```
void Adc_Setup (void);
```

```
void IO_Setup (void);
```

```
void Led_Blink (void);
```

```
void main (void)
```

```
{
```

```
    WDTCTL = WDTPW + WDTHOLD;
```

```
// STOP
```

```
    WATCHDOG TIMER TO PREVENT TIME OUT RESET
```

```
    while (1)
```

```
    {
```

```
        switch (Status_Machine)
```

```
        {
```

```
            case SM_Hardware_Setup:
```

```
                Hardware_Setup();
```

```
                Status_Machine = SM_Led_Blink;
```

```
                break;
```

```
            case SM_Led_Blink:
```

```
                Led_Blink();
```

```
                break;
```

```
        }
```

```
    }
```

```
}
```

```
void Hardware_Setup (void)
```

```
{
```

```

Oscilator_Setup();
Adc_Setup();
Timer_Setup();
IO_Setup();

__enable_interrupt();           // HABILITA AS
INTERRUPCOES
}

void Oscilator_Setup (void)
{
    BCSCCTL1 = CALBC1_8MHZ;           //
    CONFIGURA O DCO PARA 8 MHz
    DCOCTL = CALDCO_8MHZ;
}

void Timer_Setup (void)
{
    // CONFIGURA O TIMER
    TA0CCTL0 = CCIE;
    TA0CTL = TASSEL_2 + MC_1 + ID_2;
    TA0CCR0 = 2000;
    TA0CTL &= ~TAIFG;

    // __enable_interrupt();           // HABILITA AS
    INTERRUPCOES
}

void Adc_Setup (void)
{
    // CONFIGURAR ADC
    ADC10CTL0 &= ~ENC;           // DESABILITA O
    ADC PARA CONFIGURACAO
    ADC10CTL1 = INCH_0 | ADC10DIV_0 | CONSEQ_0;           //
    CONFIGURA O ADC PARA CANAL A0, MODO DE AMOSTRAGEM UNICA,
    DIVISAO DO CLOCK ADC10CLOCK/1
    ADC10CTL0 = SREF_0 | ADC10SHT_2 | ADC10ON;           //
    CONFIGURA O ADC PARA RESOLUCAO DE 10 BITS E MODO DE
    CONVERSAO UNICA
    ADC10AE0 |= (BIT4|BIT5);
}

void IO_Setup (void)
{
    P1DIR |= LED_VD;           // CONFIGURACAO
    DOS PINOS DOS LEDS COMO SAIDA
    P1OUT &= ~LED_VD;           //
    CONFIGURACAO DOS PINOS DOS LEDS COMO ENTRADA

```

```

}

void Led_Blink (void)
{
    if(Led_Blink_Tmr < 700 )
    {
        P1OUT &= ~LED_VD;           // DESLIGA O
        LED_VD
    }
    else
    {
        if (Led_Blink_Tmr <= 999)
        {
            P1OUT |= LED_VD;       // LIGA O LED_VD
        }
        else
        {
            Led_Blink_Tmr =0;
        }
    }
}

```

```

// SERVICO DA ROTINA DE INTERRUPTAO DO TIMER
#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A (void)
{
    TA0CTL &= ~TAIFG;           // LIMPA A FLAG
    DE INTERRUPTAO
    Led_Blink_Tmr++;
}

```

PARTE B

```

#include <msp430g2553.h>
#include <math.h>

```

// DEFINE

```

#define SM_Hardware_Setup           (0)
#define SM_Led_Blink                (1)
#define SM_Read_ADC                 (2)
#define analogica_temperatura        (INCH_4)
#define analogica_iluminancia        (INCH_10)

```

```

#define LED_VD                       (BIT0)
#define NUM_AMOSTRAS                  (512)

```

// DECLARACAO DAS VARIAVEIS

```

unsigned char Status_Machine = SM_Hardware_Setup;
unsigned int Led_Blink_Tmr = 0;

```

```
unsigned int Read_ADC_Tmr = 0;
unsigned int Contador = 0;
int Temperatura_Total = 0;
int Iluminancia_Total = 0;
unsigned int Media_Temperatura;
unsigned int Media_Iluminancia;
float Temperatura;
float Iluminancia;
```

```
// DECLARACAO DOS PROTOTIPOS
```

```
void Hardware_Setup (void);
void Oscilator_Setup (void);
void Timer_Setup (void);
void Adc_Setup (void);
void IO_Setup (void);
void Led_Blink (void);
void Read_ADC_Temperatura (void);
void Read_ADC_Iluminancia (void);
float Calcular_Temperatura (float ADC);
float Calcular_Iluminancia (float ADC);
```

```
void main (void)
```

```
{
    WDTCTL = WDTPW + WDTHOLD; // STOP
    WATCHDOG TIMER TO PREVENT TIME OUT RESET
```

```
while (1)
```

```
{
    switch (Status_Machine)
    {
        case SM_Hardware_Setup:

            Hardware_Setup();

            Status_Machine = SM_Led_Blink;

            break;

        case SM_Led_Blink:
            Led_Blink();
            Status_Machine = SM_Read_ADC;

        case SM_Read_ADC:
            if (Read_ADC_Tmr >=1)
            {
                // LEITURA DOS SENSORES
                Read_ADC_Temperatura();
```

```

Read_ADC_Iluminancia();

Contador++;

if(Contador >= NUM_AMOSTRAS)
{
    Media_Temperatura= Temperatura_Total/NUM_AMOSTRAS;
    Media_Iluminancia= Iluminancia_Total/NUM_AMOSTRAS;

    Temperatura= Calcular_Temperatura(Media_Temperatura);
    Iluminancia= Calcular_Iluminancia(Media_Iluminancia);

    // REINICIA AS VARIÁVEIS
    Contador = 0;
    Temperatura_Total = 0;
    Iluminancia_Total = 0;
}
Read_ADC_Tmr = 0; // Reinicia o
temporizador do ADC
Status_Machine = SM_Led_Blink;
}
break;
}
}
}

void Hardware_Setup (void)
{
    Oscilator_Setup();
    Adc_Setup();
    Timer_Setup();
    IO_Setup();

    __enable_interrupt(); // HABILITA AS
    INTERRUPTCOES
}

void Oscilator_Setup (void)
{
    BCSCTL1 = CALBC1_8MHZ; //
    CONFIGURA O DCO PARA 8 MHz
    DCOCTL = CALDCO_8MHZ;
}

void Timer_Setup (void)
{
    // CONFIGURA O TIMER
    TA0CCTL0 = CCIE;

```

```

TA0CTL = TASSEL_2 + MC_1 + ID_2;
TA0CCR0 = 2000;
TA0CTL &= ~TAIFG;

// __enable_interrupt();
// HABILITA AS
// INTERRUPT COES
}

void Adc_Setup (void)
{
    // CONFIGURAR ADC
    ADC10CTL0 &= ~ENC;
    // DESABILITA O
    // ADC PARA CONFIGURACAO
    ADC10CTL1 = INCH_0 | ADC10DIV_0 | CONSEQ_0;
    //
    // CONFIGURA O ADC PARA CANAL A0, MODO DE AMOSTRAGEM UNICA,
    // DIVISAO DO CLOCK ADC10CLOCK/1
    ADC10CTL0 = SREF_0 | ADC10SHT_2 | ADC10ON;
    //
    // CONFIGURA O ADC PARA RESOLUCAO DE 10 BITS E MODO DE
    // CONVERSAO UNICA
    ADC10AE0 |= (BIT4|BIT5);
}

void IO_Setup (void)
{
    P1DIR |= LED_VD;
    // CONFIGURACAO
    // DOS PINOS DOS LEDS COMO SAIDA
    P1OUT &= ~LED_VD;
    //
    // CONFIGURACAO DOS PINOS DOS LEDS COMO ENTRADA
}

void Led_Blink (void)
{
    if(Led_Blink_Tmr < 700 )
    {
        P1OUT &= ~LED_VD;
        // DESLIGA O
        // LED_VD
    }
    else
    {
        if (Led_Blink_Tmr <= 999)
        {
            P1OUT |= LED_VD;
            // LIGA O LED_VD
        }
        else
        {
            Led_Blink_Tmr =0;
        }
    }
}

```

```
}
```

```
void Read_ADC_Temperatura (void)
```

```
{
```

```
    ADC10CTL0 &= ~ENC;
```

```
// DESABILITA A
```

```
    CONFIGURACAO DO ADC
```

```
    ADC10CTL1 = analogica_temperatura;
```

```
//
```

```
    SELECIONA O CANAL
```

```
    ADC10CTL0 |= ENC + ADC10SC;
```

```
// INICIA A
```

```
    CONVERSAO
```

```
    while (ADC10CTL1 & ADC10BUSY);
```

```
//
```

```
    AGUARDA A CONVERSAO SER CONCLUIDA
```

```
    Temperatura_Total += ADC10MEM;
```

```
// LE O
```

```
    VALOR CONVERTIDO
```

```
    ADC10CTL0 &= ~ENC;
```

```
// TERMINA A
```

```
    CONVERSAO
```

```
}
```

```
void Read_ADC_Iluminancia (void)
```

```
{
```

```
    ADC10CTL0 &= ~ENC;
```

```
// DESABILITA A
```

```
    CONFIGURACAO DO ADC
```

```
    ADC10CTL1 = analogica_iluminancia;
```

```
//
```

```
    SELECIONA O CANAL
```

```
    ADC10CTL0 |= ENC + ADC10SC;
```

```
// INICIA A
```

```
    CONVERSAO
```

```
    while (ADC10CTL1 & ADC10BUSY);
```

```
//
```

```
    AGUARDA A CONVERSAO SER CONCLUIDA
```

```
    Iluminancia_Total += ADC10MEM;
```

```
// L O
```

```
    VALOR CONVERTIDO
```

```
    ADC10CTL0 &= ~ENC;
```

```
// TERMINA A
```

```
    CONVERSAO
```

```
}
```

```
// CALCULO TEMPERATURA
```

```
float Calcular_Temperatura (float ADC)
```

```
{
```

```
    ADC= (ADC*1023)/3.3;
```

```
    ADC= (ADC-0.986)/0.035;
```

```
    return ADC;
```

```
}
```

```
// CALCULO ILUMINANCIA
```

```
float Calcular_Iluminancia (float ADC)
```

```
{
```

```
    ADC= (ADC*1023)/3.3;
```

```
    ADC= (1000*(3.3-ADC))/ADC;
```

```
    ADC= pow(ADC,-4.0/3.0)*pow(10,20/3);
```

```

    return ADC;
}

// SERVICO DA ROTINA DE INTERRUPTAO DO TIMER
#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A (void)
{
    TA0CTL &= ~TAIFG; // LIMPA A FLAG
    DE INTERRUPTAO
    Led_Blink_Tmr++;
    Read_ADC_Tmr++;
}

```

PARTE C

```

#include <msp430g2553.h>
#include <math.h>

```

```

// DEFINE
#define SM_Hardware_Setup (0)
#define SM_Led_Blink (1)
#define SM_Read_ADC (2)
#define analogica_temperatura (INCH_4)
#define analogica_iluminancia (INCH_10)

#define LED_VD (BIT0)
#define NUM_AMOSTRAS (512)
#define LUT_Size (256)

```

```

// DECLARACAO DAS VARIAVEIS
unsigned char Status_Machine = SM_Hardware_Setup;
unsigned int Led_Blink_Tmr = 0;
unsigned int Read_ADC_Tmr = 0;
unsigned int Contador = 0;
int Temperatura_Total = 0;
int Iluminancia_Total = 0;
unsigned int Media_Temperatura;
unsigned int Media_Iluminancia;
float Temperatura;
float Iluminancia;

```

```

// DECLARACAO DOS PROTOTIPOS
void Hardware_Setup (void);
void Oscilator_Setup (void);
void Timer_Setup (void);
void Adc_Setup (void);
void IO_Setup (void);
void Led_Blink (void);
void Read_ADC_Temperatura (void);

```



```
void Read_ADC_Iluminancia (void);  
float Calcular_Temperatura (float ADC);  
float Calcular_Iluminancia (float ADC);
```

```
const int sino_LUT[LUT_Size]={0, 804, 1608, 2410,3212, 4011, 4808,  
5602, 6393, 7179, 7962, 8739, 9512, 10278,  
11039, 11793, 12539, 13279, 14010, 14732, 15446,  
16151, 16846, 17530, 18204, 18868, 19519, 20159,  
20787, 21403, 22005, 22594, 23170, 23731, 24279,  
24811, 25329, 25832, 26319, 26790, 27245, 27683,  
28105, 28510, 28898, 29268, 29621, 29956, 30273,  
30571, 30852, 31113, 31356, 31580, 31785, 31971,  
32137, 32285, 32412, 32521, 32609, 32678, 32728,  
32757, 32767, 32757, 32728, 32678, 32609, 32521,  
32412, 32285, 32137, 31971, 31785, 31580, 31356,  
31113, 30852, 30571, 30273, 29956, 29621, 29268,  
28898, 28510, 28105, 27683, 27245, 26790, 26319,  
25832, 25329, 24811, 24279, 23731, 23170, 22594,  
22005, 21403, 20787, 20159, 19519, 18868, 18204,  
17530, 16846, 16151, 15446, 14732, 14010, 13279,  
12539, 11793, 11039, 10278, 9512, 8739, 7962,  
7179, 6393, 5602, 4808, 4011, 3212, 2410,  
1608, 804, 0, -804, -1608, -2410, -3212,  
-4011, -4808, -5602, -6393, -7179, -7962, -8739,  
-9512, -10278, -11039, -11793, -12539, -13279, -14010,  
-14732, -15446, -16151, -16846, -17530, -18204, -18868,  
-19519, -20159, -20787, -21403, -22005, -22594, -23170,  
-23731, -24279, -24811, -25329, -25832, -26319, -26790,  
-27245, -27683, -28105, -28510, -28898, -29268, -29621,  
-29956, -30273, -30571, -30852, -31113, -31356, -31580,  
-31785, -31971, -32137, -32285, -32412, -32521, -32609,  
-32678, -32728, -32757, -32767, -32757, -32728, -32678,  
-32609, -32521, -32412, -32285, -32137, -31971, -31785,  
-31580, -31356, -31113, -30852, -30571, -30273, -29956,  
-29621, -29268, -28898, -28510, -28105, -27683, -27245,  
-26790, -26319, -25832, -25329, -24811, -24279, -23731,  
-23170, -22594, -22005, -21403, -20787, -20159, -19519,  
-18868, -18204, -17530, -16846, -16151, -15446, -14732,  
-14010, -13279, -12539, -11793, -11039, -10278, -9512,  
-8739, -7962, -7179, -6393, -5602, -4808, -4011,  
-3212, -2410, -1608, -804 };
```

```
void main (void)  
{  
    WDTCTL = WDTPW + WDTHOLD; // STOP  
WATCHDOG TIMER TO PREVENT TIME OUT RESET  
  
    while (1)
```

```

{
    switch (Status_Machine)
    {
    case SM_Hardware_Setup:

        Hardware_Setup();

        Status_Machine = SM_Led_Blink;

        break;

    case SM_Led_Blink:
        Led_Blink();
        Status_Machine = SM_Read_ADC;

    case SM_Read_ADC:
        if (Read_ADC_Tmr >=1)
        {
            // LEITURA DOS SENSORES
            Read_ADC_Temperatura();
            Read_ADC_Iluminancia();

            Contador++;

            if(Contador >= NUM_AMOSTRAS)
            {
                Media_Temperatura= Temperatura_Total/NUM_AMOSTRAS;
                Media_Iluminancia= Iluminancia_Total/NUM_AMOSTRAS;

                Temperatura= Calcular_Temperatura(Media_Temperatura);
                Iluminancia= Calcular_Iluminancia(Media_Iluminancia);

                // REINICIA AS VARIÁVEIS
                Contador = 0;
                Temperatura_Total = 0;
                Iluminancia_Total = 0;
            }
            Read_ADC_Tmr = 0; // Reinicia o
temporizador do ADC
            Status_Machine = SM_Led_Blink;
        }
        break;
    }
}

void Hardware_Setup (void)
{

```

```

Oscilator_Setup();
Adc_Setup();
Timer_Setup();
IO_Setup();

__enable_interrupt();           // HABILITA AS
INTERRUPCOES
}

void Oscilator_Setup (void)
{
    BCSCCTL1 = CALBC1_8MHZ;           //
    CONFIGURA O DCO PARA 8 MHz
    DCOCTL = CALDCO_8MHZ;
}

void Timer_Setup (void)
{
    // CONFIGURA O TIMER
    TA0CCTL0 = CCIE;
    TA0CTL = TASSEL_2 + MC_1 + ID_2;
    TA0CCR0 = 2000;
    TA0CTL &= ~TAIFG;

    // __enable_interrupt();           // HABILITA AS
    INTERRUPCOES
}

void Adc_Setup (void)
{
    // CONFIGURAR ADC
    ADC10CTL0 &= ~ENC;           // DESABILITA O
    ADC PARA CONFIGURACAO
    ADC10CTL1 = INCH_0 | ADC10DIV_0 | CONSEQ_0;           //
    CONFIGURA O ADC PARA CANAL A0, MODO DE AMOSTRAGEM UNICA,
    DIVISAO DO CLOCK ADC10CLOCK/1
    ADC10CTL0 = SREF_0 | ADC10SHT_2 | ADC10ON;           //
    CONFIGURA O ADC PARA RESOLUCAO DE 10 BITS E MODO DE
    CONVERSAO UNICA
    ADC10AE0 |= (BIT4|BIT5);
}

void IO_Setup (void)
{
    P1DIR |= LED_VD;           // CONFIGURACAO
    DOS PINOS DOS LEDS COMO SAIDA
    P1OUT &= ~LED_VD;           //
    CONFIGURACAO DOS PINOS DOS LEDS COMO ENTRADA

```

```

}

void Led_Blink (void)
{
    if(Led_Blink_Tmr < 700 )
    {
        P1OUT &= ~LED_VD;           // DESLIGA O
        LED_VD
    }
    else
    {
        if (Led_Blink_Tmr <= 999)
        {
            P1OUT |= LED_VD;       // LIGA O LED_VD
        }
        else
        {
            Led_Blink_Tmr =0;
        }
    }
}

```

```

void Read_ADC_Temperatura (void)
{
    ADC10CTL0 &= ~ENC;           // DESABILITA A
    CONFIGURACAO DO ADC
    ADC10CTL1 = analogica_temperatura; //
    SELECIONA O CANAL
    ADC10CTL0 |= ENC + ADC10SC;   // INICIA A
    CONVERSAO
    while (ADC10CTL1 & ADC10BUSY); //
    AGUARDA A CONVERSAO SER CONCLUIDA
    Temperatura_Total += ADC10MEM; // LE O
    VALOR CONVERTIDO
    ADC10CTL0 &= ~ENC;           // TERMINA A
    CONVERSAO
}

```

```

void Read_ADC_Iluminancia (void)
{
    ADC10CTL0 &= ~ENC;           // DESABILITA A
    CONFIGURACAO DO ADC
    ADC10CTL1 = analogica_iluminancia; //
    SELECIONA O CANAL
    ADC10CTL0 |= ENC + ADC10SC;   // INICIA A
    CONVERSAO
    while (ADC10CTL1 & ADC10BUSY); //
    AGUARDA A CONVERSAO SER CONCLUIDA
}

```

```

    Iluminancia_Total += ADC10MEM;                                // L O
    VALOR CONVERTIDO
    ADC10CTL0 &= ~ENC;                                           // TERMINA A
    CONVERSAO
}

// CALCULO TEMPERATURA
float Calcular_Temperatura (float ADC)
{
    ADC= (ADC*1023)/3.3;
    ADC= (ADC-0.986)/0.035;
    return ADC;
}

// CALCULO ILUMINANCIA
float Calcular_Iluminancia (float ADC)
{
    ADC= (ADC*1023)/3.3;
    ADC= (1000*(3.3-ADC))/ADC;
    ADC= pow(ADC,-4.0/3.0)*pow(10,20/3);
    return ADC;
}

// SERVICO DA ROTINA DE INTERRUPCAO DO TIMER
#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A (void)
{
    TA0CTL &= ~TAIFG;                                           // LIMPA A FLAG
    DE INTERRUPCAO
    Led_Blink_Tmr++;
    Read_ADC_Tmr++;
}

PARTE D
#include <msp430g2553.h>
#include <math.h>

// DEFINE
#define SM_Hardware_Setup (0)
#define SM_Led_Blink (1)
#define SM_Read_ADC (2)
#define analogica_temperatura (INCH_4)
#define analogica_iluminancia (INCH_10)

#define LED_VD (BIT0)
#define NUM_AMOSTRAS (512)
#define LUT_Size (256)

```

```
// DECLARACAO DAS VARIAVEIS
```

```
unsigned char Status_Machine = SM_Hardware_Setup;
```

```
unsigned int Led_Blink_Tmr = 0;
```

```
unsigned int Read_ADC_Tmr = 0;
```

```
unsigned int Contador = 0;
```

```
int Temperatura_Total = 0;
```

```
int Iluminancia_Total = 0;
```

```
unsigned int Media_Temperatura;
```

```
unsigned int Media_Iluminancia;
```

```
float Temperatura;
```

```
float Iluminancia;
```

```
// DECLARACAO DOS PROTOTIPOS
```

```
void Hardware_Setup (void);
```

```
void Oscilator_Setup (void);
```

```
void Timer_Setup (void);
```

```
void Adc_Setup (void);
```

```
void IO_Setup (void);
```

```
void Led_Blink (void);
```

```
void Read_ADC_Temperatura (void);
```

```
void Read_ADC_Iluminancia (void);
```

```
float Calcular_Temperatura (float ADC);
```

```
float Calcular_Iluminancia (float ADC);
```

```
int Send_Dados (void);
```

```
void UART_Setup (void);
```

```
void Convert_To_Str (void);
```

```
const int sino_LUT[LUT_Size]={0, 804, 1608, 2410,3212, 4011, 4808,  
5602, 6393, 7179, 7962, 8739, 9512, 10278,  
11039, 11793, 12539, 13279, 14010, 14732, 15446,  
16151, 16846, 17530, 18204, 18868, 19519, 20159,  
20787, 21403, 22005, 22594, 23170, 23731, 24279,  
24811, 25329, 25832, 26319, 26790, 27245, 27683,  
28105, 28510, 28898, 29268, 29621, 29956, 30273,  
30571, 30852, 31113, 31356, 31580, 31785, 31971,  
32137, 32285, 32412, 32521, 32609, 32678, 32728,  
32757, 32767, 32757, 32728, 32678, 32609, 32521,  
32412, 32285, 32137, 31971, 31785, 31580, 31356,  
31113, 30852, 30571, 30273, 29956, 29621, 29268,  
28898, 28510, 28105, 27683, 27245, 26790, 26319,  
25832, 25329, 24811, 24279, 23731, 23170, 22594,  
22005, 21403, 20787, 20159, 19519, 18868, 18204,  
17530, 16846, 16151, 15446, 14732, 14010, 13279,  
12539, 11793, 11039, 10278, 9512, 8739, 7962,  
7179, 6393, 5602, 4808, 4011, 3212, 2410,  
1608, 804, 0, -804, -1608, -2410, -3212,  
-4011, -4808, -5602, -6393, -7179, -7962, -8739,  
-9512, -10278, -11039, -11793, -12539, -13279, -14010,
```

```

-14732,-15446,-16151,-16846,-17530,-18204,-18868,
-19519,-20159,-20787,-21403,-22005,-22594,-23170,
-23731,-24279,-24811,-25329,-25832,-26319,-26790,
-27245,-27683,-28105,-28510,-28898,-29268,-29621,
-29956,-30273,-30571,-30852,-31113,-31356,-31580,
-31785,-31971,-32137,-32285,-32412,-32521,-32609,
-32678,-32728,-32757,-32767,-32757,-32728,-32678,
-32609,-32521,-32412,-32285,-32137,-31971,-31785,
-31580,-31356,-31113,-30852,-30571,-30273,-29956,
-29621,-29268,-28898,-28510,-28105,-27683,-27245,
-26790,-26319,-25832,-25329,-24811,-24279,-23731,
-23170,-22594,-22005,-21403,-20787,-20159,-19519,
-18868,-18204,-17530,-16846,-16151,-15446,-14732,
-14010,-13279,-12539,-11793,-11039,-10278, -9512,
-8739, -7962, -7179, -6393, -5602, -4808, -4011,
-3212, -2410, -1608, -804 };

```

```

void main (void)
{
    WDTCTL = WDTPW + WDTHOLD;                                // STOP
WATCHDOG TIMER TO PREVENT TIME OUT RESET

    while (1)
    {
        switch (Status_Machine)
        {
            case SM_Hardware_Setup:

                Hardware_Setup();

                Status_Machine = SM_Led_Blink;

                break;

            case SM_Led_Blink:
                Led_Blink();
                Status_Machine = SM_Read_ADC;

            case SM_Read_ADC:
                if (Read_ADC_Tmr >=1)
                {
                    // LEITURA DOS SENSORES
                    Read_ADC_Temperatura();
                    Read_ADC_Iluminancia();

                    Contador++;

                    if(Contador >= NUM_AMOSTRAS)

```

```

{
    Media_Temperatura= Temperatura_Total/NUM_AMOSTRAS;
    Media_Iluminancia= Iluminancia_Total/NUM_AMOSTRAS;

    Temperatura= Calcular_Temperatura(Media_Temperatura);
    Iluminancia= Calcular_Iluminancia(Media_Iluminancia);

    // REINICIA AS VARIÁVEIS
    Contador = 0;
    Temperatura_Total = 0;
    Iluminancia_Total = 0;
}
    Read_ADC_Tmr = 0; // Reinicia o
temporizador do ADC
    Status_Machine = SM_Led_Blink;
}
    break;
}
}
}

void Hardware_Setup (void)
{
    Oscilator_Setup();
    Adc_Setup();
    Timer_Setup();
    IO_Setup();

    __enable_interrupt(); // HABILITA AS
    INTERRUPTCOES
}

void Oscilator_Setup (void)
{
    BCSCTL1 = CALBC1_8MHZ; //
    CONFIGURA O DCO PARA 8 MHz
    DCOCTL = CALDCO_8MHZ;
}

void Timer_Setup (void)
{
    // CONFIGURA O TIMER
    TA0CCTL0 = CCIE;
    TA0CTL = TASSEL_2 + MC_1 + ID_2;
    TA0CCR0 = 2000;
    TA0CTL &= ~TAIFG;
}

```



```

// __enable_interrupt();
// HABILITA AS
INTERRUPTOS
}

void Adc_Setup (void)
{
    // CONFIGURAR ADC
    ADC10CTL0 &= ~ENC;
    // DESABILITA O
    ADC PARA CONFIGURACAO
    ADC10CTL1 = INCH_0 | ADC10DIV_0 | CONSEQ_0;
    //
    CONFIGURA O ADC PARA CANAL A0, MODO DE AMOSTRAGEM UNICA,
    DIVISAO DO CLOCK ADC10CLOCK/1
    ADC10CTL0 = SREF_0 | ADC10SHT_2 | ADC10ON;
    //
    CONFIGURA O ADC PARA RESOLUCAO DE 10 BITS E MODO DE
    CONVERSAO UNICA
    ADC10AE0 |= (BIT4|BIT5);
}

void IO_Setup (void)
{
    P1DIR |= LED_VD;
    // CONFIGURACAO
    DOS PINOS DOS LEDS COMO SAIDA
    P1OUT &= ~LED_VD;
    //
    CONFIGURACAO DOS PINOS DOS LEDS COMO ENTRADA
}

void UART_Setup (void)
{
    // SELECT PORTS P1.1 (RX) AND P1.2 (TX) FOR THE UART FUNCTION
    P1SEL |= BIT1 + BIT2;
    P1SEL2 |= BIT1 + BIT2;

    UCA0CTL1 |= UCSSEL_2;
    // SELECT THE
    SMCLK AS CLOCK SOURCE
    UCA0BR0 = 0x41;
    // SETS THE BAUD
    RATE TO 9600
    UCA0BR1 = 0x03;
    UCA0MCTL = UCBRS0;
    //
    CONFIGURES THE BAUD RATE MODULATION
    UCA0CTL1 &= ~UCSWRST;
    // RELEASES
    THE INTERFACE FOR OPERATION
    UCOIE |= UCA0RXIE;
    // ENABLE
    USCI_A0 RX INTERRUPT
    //UCOIE |= UCA0TXIE;
    // ENABLE
    USCI_A0 TX INTERRUPT
}

void Led_Blink (void)

```

```

{
    if(Led_Blink_Tmr < 700 )
    {
        P1OUT &= ~LED_VD;           // DESLIGA O
        LED_VD
    }
    else
    {
        if (Led_Blink_Tmr <= 999)
        {
            P1OUT |= LED_VD;       // LIGA O LED_VD
        }
        else
        {
            Led_Blink_Tmr =0;
        }
    }
}

```

```

void Read_ADC_Temperatura (void)
{
    ADC10CTL0 &= ~ENC;           // DESABILITA A
    CONFIGURACAO DO ADC
    ADC10CTL1 = analogica_temperatura; //
    SELECIONA O CANAL
    ADC10CTL0 |= ENC + ADC10SC;  // INICIA A
    CONVERSAO
    while (ADC10CTL1 & ADC10BUSY); //
    AGUARDA A CONVERSAO SER CONCLUIDA
    Temperatura_Total += ADC10MEM; // LE O
    VALOR CONVERTIDO
    ADC10CTL0 &= ~ENC;           // TERMINA A
    CONVERSAO
}

```

```

void Read_ADC_Iluminancia (void)
{
    ADC10CTL0 &= ~ENC;           // DESABILITA A
    CONFIGURACAO DO ADC
    ADC10CTL1 = analogica_iluminancia; //
    SELECIONA O CANAL
    ADC10CTL0 |= ENC + ADC10SC;  // INICIA A
    CONVERSAO
    while (ADC10CTL1 & ADC10BUSY); //
    AGUARDA A CONVERSAO SER CONCLUIDA
    Iluminancia_Total += ADC10MEM; // L O
    VALOR CONVERTIDO
}

```

```

    ADC10CTL0 &= ~ENC;                                     // TERMINA A
CONVERSAO
}

// CALCULO TEMPERATURA
float Calcular_Temperatura (float ADC)
{
    ADC= (ADC*1023)/3.3;
    ADC= (ADC-0.986)/0.035;
    return ADC;
}

// CALCULO ILUMINANCIA
float Calcular_Iluminancia (float ADC)
{
    ADC= (ADC*1023)/3.3;
    ADC= (1000*(3.3-ADC))/ADC;
    ADC= pow(ADC,-4.0/3.0)*pow(10,20/3);
    return ADC;
}

// SERVICO DA ROTINA DE INTERRUPTAO DO TIMER
#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A (void)
{
    TA0CTL &= ~TAIFG;                                     // LIMPA A FLAG
DE INTERRUPTAO
    Led_Blink_Tmr++;
    Read_ADC_Tmr++;
}

//FUNCAO PARA ENVIAR OS DADOS PARA UART
int Send_Dados (void)
{
    int j=6, i=0, a=0;                                     // ARMAZENA O
VALOR DE sino_LUT[i] EM UMA VARIABEL TEMPORARIA
    char car[6];

    WDTCTL = WDTPW + WDTHOLD;                             // STOP
WATCHDOG TIME TO PREVENT TIME OUT RESET

    while(1)
    {
        if (i >= LUT_Size)
        {
            i=0;                                           //GARANTE QUE O i NAO
ULTRAPASSE OS LIMITES DO ARRAY, OU SEJA, HA UM RESET NO
VALOR DE i SEMPRE QUE EXCEDER O TAMANHO DA LUT

```

```

    }

    int SENO= sino_LUT[i++]; //ARMAZENA O
    VALOR ATUAL DE sino_LUT[i] EM temperatura e incrementa i
    for (j=6; j>=0; j--)

    {
        car[j] = 0; //REINICIA A STRING
        caracter PARA 0
    }

    j=5; //j ... INICIALIZADO A 5,
    INDICANDO QUE VAMOS COMECAR A PREENCHER A STRING caracter DE
    TRAS PARA FRENTE (FINAL PARA O INICIO)

    while(Temperatura > 0) //CONVERSAO
    DOS VALORES INTEIROS PARA STRING
    {
        a= SENO %10; //EXTRAIR O DIGITO
        MENOS SIGNIFICATIVO ENQUANTO temperatura > 0
        SENO = SENO/10; //REMOVE O DIGITO
        MENOS SIGNIFICATIVO DE temperatura
        car[j--] = 48 + a; //CONVERTE O DIGITO
        PARA SEU EQUIVALENTE EM CARACTER ASCII (ADICIONANDO 48) E
        ARMAZENANDO NA POSICAO j, DEPOIS DECREMENTA j
    }

    car[6] = '\0'; //ADICIONA O CARACTER
    NULO NO FINAL DA STRING
}

//PASSAR DE INT PARA STRING (TEMPERATURA E ILUMINANCIA)
void Convert_To_Str (void)
{
    int k = 0;
    int kNegative = 0;

    //NUMEROS NEGATIVOS
    if (num<0)
    {
        kNegative = 1;
        num = -num;
    }

    while(num>0)
    {
        str[k++] = (num % 10) + '0';
    }

```

```
    num /= 10;
}

if (kNegative)
{
    str[k++] = '-';
}

str[k] = '\0';
int start = 0;
int end = k - 1;
while (start < end)
{
    char temp = str[start];
    str[start] = str[end];
    str[end] = temp;
    start++;
    end--;
}
}
```